Filtering, Sorting, Aggregating, and Binning Using SQL

Explore how to use SQL for filtering, sorting, aggregating, and binning data to transform large data sets. Understand how these methods prepare data by removing irrelevant entries, summarizing trends, and categorizing values to enhance analysis and loading to data marts.

Filtering, sorting, aggregating, and binning are other commonly used operations to transform data before loading it to its final destination for analysis. Let's discuss how we can use these operations through a use case.

Use case: Analyzing flight delay data

As employees of an airline, we're tasked with transferring relevant flight data from the company's cloud data warehouse to a new data mart.

Because of recent flight delays, the company has reserved a data mart for this task. The data mart will assist the analysts in trying to identify the root cause of the recent flight delays. Our task is to transform the huge data set collected by the airline company in the cloud data warehouse and keep only relevant data for the analysts.

For example, we can remove flights that were on time from the data set and therefore irrelevant for this use case.

Here's the sample flight data from the data warehouse:

Ask

main.sql PostgreSQL

```sql
-- Select the first 5 rows
select
    id as flight_id,
    exp_dep as expected_departure,
    exp_arr as expected_arrival,
    departure as actual_departure,
    arrival as actual_arrival,
from flight_data
limit 5;
```

Run

Sample flight data

Filtering

Filtering is the process of removing unwanted data according to certain conditions. Filtering in SQL can be performed using conditional statements like `where` and `case`. For example, we can remove from the data set the flights that were on time and are, therefore, irrelevant to this use case. Let's create a `view` of all the delayed flights.

A `view` **in SQL** is just a named query, and it's not stored in the disk the same way tables are. Therefore, they are a great tool when performing the same query repeatedly.

flight_data2.sql PostgreSQL

```sql
-- create a view of all delayed flights
create view delayed_flights as
select *
from flight_data
where
    departure > exp_dep;

-- Select the first 5 delayed flights
```

Ask

```
select
    id as flight_id,
    exp_dep as expected_departure,
    exp_arr as expected_arrival,
    departure as actual_departure
```

Run

Creating a view of all delayed flights

Sorting

Sorting is more of a cosmetical transformation rather than a logical one. Nevertheless, it can help us detect patterns in the data and is crucial when plotting time series data like this flight data. Let's improve the previous `view`. This time, we'll sort the data by `expected_departure` using the `order by` statement.

flight_data3.sql PostgreSQL

```
-- create a view of all delayed flights. Order by expected departures
create or replace view delayed_flights as
select *
from flight_data
where
    departure > exp_dep
order by exp_dep desc;

-- Select the first 10 delayed flights
select
    id as flight_id,
    exp_dep as expected_departure,
    exp_arr as expected_arrival
```

Run

Sort all delayed flights by expected departure

Ask

Aggregating

Aggregating is the process of summarizing data into a single value. Aggregating records/columns into a single value can help us understand the general picture of how a data set behaves. For example, aggregating an array of values into a single value, such as standard deviation, will allow us to understand how far, on average, each data point is from the mean.

For this demonstration, we'll add a column to the `view` that contains the difference in minutes between the `expected_departure` and the `actual_departure` of each delayed flight. Then we can add a moving average to the `view` to showcase the trend of the average delay time. Doing this will make the analyst's job of analyzing and plotting the data much more straightforward.

flight_data4.sql PostgreSQL

```
-- Create a view of all delayed flights. calculate delay in minutes and a
create or replace view delayed_flights as
select
    aaa.*,
    round(
         (delay_in_minutes +
         (LAG(delay_in_minutes, 2) over(ORDER BY exp_dep )) +
         (LAG(delay_in_minutes, 1) over(ORDER BY exp_dep ))) / 3
         ,2
    ) as Moving_Average_3
from
(
    select
```

Run

View of aggregate process of all delayed flights

Binning

Binning is grouping data into discrete categories or "bins." This method is a way to convert a continuous array of values into a discrete one.

Ask

For this demonstration, we can add to `view` the categories for the severity of the flight delay by binning the column `delay_in_minutes`. The idea is that the analysts might want to focus on the extreme cases of flight delays first before attending to the rest. This can be a great way to allow them to filter the data according to the severity of the delay.

flight_data5.sql PostgreSQL

```
-- Creating a view of all delayed flight. Adding delay_severity
create or replace view delayed_flights as
select
    aaa.*,
    round(
         (delay_in_minutes +
         (LAG(delay_in_minutes, 2) over(ORDER BY exp_dep )) +
         (LAG(delay_in_minutes, 1) over(ORDER BY exp_dep ))) / 3
         ,2
    ) as Moving_Average_3,
    case
        when delay_in_minutes > 40 then 'Long_Delay'
        when delay_in_minutes between 15 and 40 then 'Medium_Delay'
```

Run

Adding the column: delay_severity

After creating a `view` with all the information required to analyze the data and identify the root cause, we can export the `view` as a CSV file and load it to the company's data mart. Now, the analysts will have a specialized repository to query delayed flight data.

main.sql PostgreSQL

```
-- Exporting the delayed_flights view
COPY (
    select *
    from delayed_flights
)
TO '/tmp/delayed_flights.csv'
```

Ask

```
    DELIMITER ','
    CSV HEADER;
```

Exporting the view as CSV

Conclusion

These transformations play a critical role in cleaning, transforming, and shaping the data for analysis and modeling purposes. They help us focus on the most relevant information and make our data more manageable and easier to work with.

Ask